NATIONAL ECONOMICS UNIVERSITY
FACULTY OF MATHEMATICAL ECONOMICS

MAJOR ASSIGNMENT

Course: Deep Learning

TOPIC:

Robust Real-World Waste Classification using Custom CNN Architectures and Transfer Learning

Instructor:

Nguyen Thi Kim Ngan

Students:

Nguyen Vinh Khanh
Do Huu Kien
Nguyen Tong Nguyen Vu
Nguyen Phu Nam

Hanoi, October 2026

Part 1: Theory

This part explains the building blocks of a convolutional neural network (CNN) that our three models are made of. We follow the order of the course slides [1, 2, 3], and for each idea we point out where it appears in our own networks, so that the theory can be read together with the model design in Part 2.

1. Convolution, filters and feature maps

A convolutional layer extracts local patterns with small kernels that are shared across the whole image, which keeps the number of weights low. A fully connected layer, by contrast, would need one weight for every pixel and colour channel, far too many for a 224×224 colour image. The convolutional layer avoids this by sliding a single small kernel, for example 3×3, over the image and computing a weighted sum at each position; the lecture defines this operation as cross-correlation [1]. Because the same weights are reused at every position, a kernel that has learned to respond to an edge finds that edge wherever it appears.

Each kernel produces one feature map, and stacking layers lets the network build complex patterns out of simple ones. A layer with 64 kernels therefore outputs 64 feature maps, which the next layer combines into patterns such as corners or textures. The size of each map depends on the padding and the stride. For an input of height n_h, a kernel of height k_h, p_h rows of padding and a stride of s_h, the output height is

$$\left\lfloor \frac{n_h - k_h + p_h + s_h}{s_h} \right\rfloor, \tag{1}$$

and the width follows the same formula [1]. All our convolutions use stride 1 and $p_h = k_h - 1$, so they keep the height and width unchanged and leave the job of shrinking the maps to the pooling layers.

2. Activation functions

Activation functions make the network non-linear; without them, a stack of convolutions would collapse into one linear map. Our own models use ReLU, $f(x) = \max(0, x)$, which is cheap to compute and keeps a non-zero gradient for every positive input, so the signal does not fade in deeper networks. EfficientNet-B0 uses SiLU, $x \cdot \sigma(x)$, a smoother version of the same idea. At the very end of the network, a different function is needed: the last layer outputs one raw score z_k per class, and

softmax turns these scores into probabilities that sum to one:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}. \tag{2}$$

3. Pooling

Pooling layers reduce the size of the feature maps and make the network less sensitive to small shifts of an object [1]. We use 2×2 max pooling with stride 2, which keeps the largest value in each window and so halves the height and width. Repeated pooling also enlarges the receptive field, that is, the area of the input image on which one value depends. This turns out to matter for our task. In the simple CNN the input shrinks to 28×28 after three poolings, whereas the multi-scale CNN reaches 7×7 after five and can therefore relate much larger parts of each object.

4. Fully connected layers and global average pooling

Fully connected layers turn the extracted features into class scores, and global average pooling lets them do so with far fewer inputs. In the classic design, the last feature maps are flattened into one long vector before the fully connected layers [1]. For our multi-scale CNN this vector would hold $7 \times 7 \times 256 = 12{,}544$ values per image. Global average pooling (GAP) instead averages every feature map into a single number and leaves only 256 values. Since GAP has no weights of its own, it also lowers the risk of overfitting. The head that follows is small: Dense(128), ReLU, Dropout and Dense(9) in our own models, and only Dropout and Dense(9) in EfficientNet-B0.

5. Loss and optimisation

All models are trained by minimising the cross-entropy loss with the AdamW optimiser. For a batch of N images with true classes y_i, the loss is

$$L = -\frac{1}{N} \sum_{i=1}^{N} \log p_{i,y_i}, \tag{3}$$

which is small when the model gives a high probability to the correct class and grows quickly when it does not. Backpropagation computes the gradient of this loss for every weight, and AdamW [9] uses these gradients to update the weights. AdamW

differs from Adam in that it applies weight decay directly to the weights instead of adding it to the gradient. We start with a learning rate of 10^{-3} and halve it whenever the validation macro-F1 has not improved for two epochs. On the GPU we also train in mixed precision (16-bit floats), which makes each epoch faster.

6. Regularisation

Because our training set is small, we combine several regularisation methods to stop the models from memorising it. The lecture lists overly complex models and small or unrepresentative datasets among the main causes of poor predictions [3], and with about 3,300 training images both risks apply to us. Batch normalisation [7, 2] follows every convolution and keeps the activations on a similar scale, which mainly makes training more stable. Dropout [8] then switches off random neurons in the head during training (30 % in our models and 20 % in EfficientNet-B0), so that the head cannot rely on any single neuron. On top of these two layers, weight decay of 10^{-4} pulls the weights towards zero, and early stopping ends training once the validation macro-F1 has not improved for five epochs.

7. Data augmentation

Data augmentation enlarges the training set by changing each image at random every time it is used, so that the model rarely sees exactly the same picture twice [3]. We chose mild changes that still produce realistic photos: a crop of 80 to 100 % of the image that is resized back to 224×224, a horizontal flip, a rotation of at most 10°, and a change of up to 20 % in brightness, contrast and saturation. The hue, however, stays unchanged, because colour is one of the cues that separate materials such as brown cardboard and grey metal. Validation and test images are only resized and normalised, so that their scores do not depend on chance.

8. Transfer learning and fine-tuning

Transfer learning reuses the knowledge of a network trained on a large dataset for a new task [3]. The lecture describes this in three phases: pretraining on a large dataset such as ImageNet, a transfer phase in which the pretrained weights are either kept fixed (feature extraction) or partly retrained, and an optional fine-tuning phase on the small target dataset [3]. Our third model follows this plan closely. We take EfficientNet-B0 pretrained on ImageNet-1K, a dataset with 1,000 classes [10], and first train only a new 9-class head on top of the frozen backbone. After that we

unfreeze the last three blocks and continue with a learning rate ten times smaller, so that the pretrained features are adjusted gently rather than overwritten.

Part 2: Practical Application

I. Problem Description

- **Title:** Robust Real-World Waste Classification using Custom CNN Architectures and Transfer Learning.
- **Research objectives:** we want to find out how much a more complex CNN and ImageNet pretraining improve the classification of real waste photos when everything else stays the same. To keep the comparison fair, all models share the same data split, preprocessing, training rules and metrics. Three research questions guide the work:

  RQ1. How well does a simple CNN trained from scratch classify RealWaste images?

  RQ2. Does our own multi-scale CNN do better than the simple CNN under the same data protocol?

  RQ3. How much does ImageNet transfer learning with EfficientNet-B0 add compared with CNNs trained from scratch?

- **Input of the problem:** one RGB photo of a single waste item, resized to 224×224 pixels.
- **Output of the problem:** the predicted class and the probability of each of the 9 classes.
- **Summary of the tasks performed:** the project moved from checking the data, through building a split without leakage, to training and comparing three models. We first audited the downloaded dataset for class counts, image format and broken files. A look at repeated photos then showed that the same object often appears in two or three consecutive images, so we replaced our first, random split with a grouped one before any model was trained. On this split we built one shared pipeline for loading, augmentation, training and evaluation, and trained the three models in Table 1, each selected by its validation macro-F1. Since a single run can be lucky or unlucky, every model was trained three times with different random seeds. Finally, each selected model was evaluated once on the test set, and we examined the images it got wrong.

Table 1: Models compared in this project.

ID	Model	Trained from	Answers
E1	SimpleCNN (ours)	scratch	RQ1
E2	MultiScaleCNN (ours)	scratch	RQ2
E3	EfficientNet-B0	ImageNet, fine-tuned	RQ3

II. Dataset Description

- **Download link:** RealWaste [4] is published in the UCI Machine Learning Repository (`https://archive.ics.uci.edu/dataset/908/realwaste`, licence CC BY 4.0). We downloaded it from the Kaggle mirror `https://www.kaggle.com/datasets/joebeachcapital/realwaste` (version 1).
- **Description:** RealWaste contains 4,752 colour photos, each showing a single waste item from one of 9 material classes: Cardboard, Food Organics, Glass, Metal, Miscellaneous Trash, Paper, Plastic, Textile Trash and Vegetation. Every image is an RGB JPEG of 524×524 pixels, and all of them opened without errors. The classes, however, are not balanced (Figure 1): Plastic has 921 images while Textile Trash has only 318, almost three times fewer. A model could therefore reach a good overall accuracy while doing badly on the small classes, which is why we compare the models mainly by macro-averaged scores.

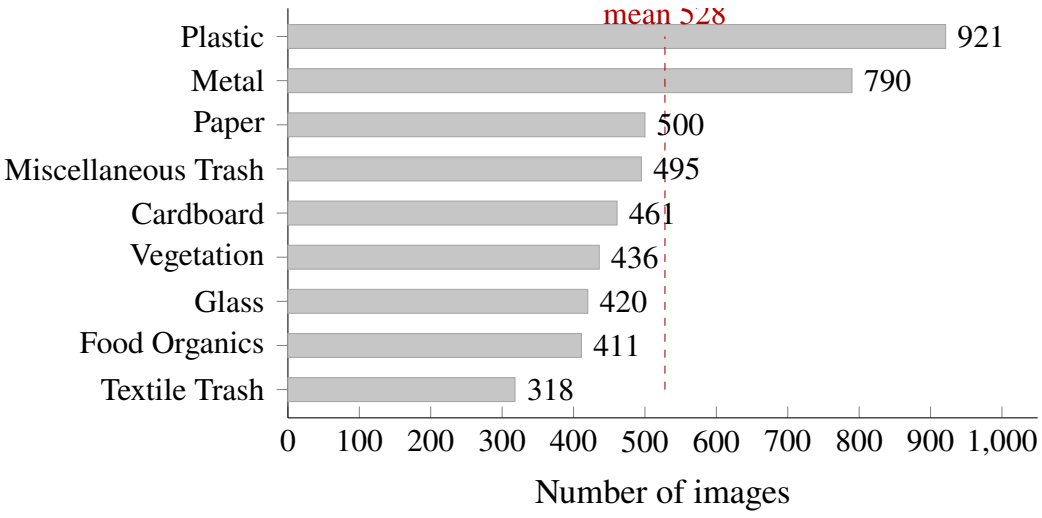


Figure 1: Images per class in our copy of RealWaste (4,752 images). Plastic has 2.9 times as many images as Textile Trash.

All photos were taken in the same setting, so the background gives the model no hint about the class. Each one shows a single item from above on the same concrete floor, which forces the model to rely on the object itself. What does vary is the lighting and the colour cast: some photos look grey, while others are clearly blue. A few items are hard to label even for a person; one Cardboard photo, for example, shows printed food packaging that we would have put under Paper or Plastic. Figure 2 shows three examples of each class.



Figure 2: Three sample images per class.

III. CNN Model Design

1. Preprocessing

Cleaning and repeated photos

The data needed no cleaning in the usual sense, but it did contain many repeated photos of the same object. All 4,752 files opened without errors and no two files were identical, so nothing had to be removed. The repeated photos only became visible when we looked at images with neighbouring file numbers. In 17 of 32 randomly chosen groups of three consecutive images, the same object had been photographed again, usually after it had been moved or turned over. Under our first, random split, 4 of these 32 objects had one photo in the training set and another in the validation or test set. A test score measured on such a split would partly reward the model for recognising objects it had already seen during training.

Grouped split

To keep all photos of one object in the same subset, we split the data by blocks of file numbers instead of by single images. Re-shots almost always have neighbouring numbers, so within each class we cut the files into blocks of ten consecutive numbers; Plastic_640 to Plastic_649, for instance, form one block. These blocks are shuffled with seed 42, and whole blocks then go to the training, validation or test set in a 70/15/15 ratio per class. Table 2 shows how much this reduced the overlap between the subsets, and Table 3 gives their final sizes.

Table 2: Overlap between subsets under the random image-level split we started with and under the grouped split we used.

Check	Random split	Grouped split
Same-class pairs 1 file number apart, in different subsets	46.7 %	4.7 %
Same-class pairs 2 file numbers apart, in different subsets	45.8 %	9.4 %
Checked objects with photos in train and in val/test	4 of 32	1 of 32

Table 3: Subsets of the grouped split (seed 42), shared by every model.

Subset	Images	Share	Purpose
Training	3,326	69.99 %	fit the weights
Validation	719	15.13 %	early stopping and checkpoint selection
Test	707	14.88 %	final evaluation, used once
Total	4,752	100 %	

The grouped split is not perfect, and we treat the remaining overlap as a limitation. Among the same 32 groups, one object still falls on both sides of a block boundary: a heart-shaped ornament photographed front and back as Miscellaneous Trash 59 and 60. Re-shots that are far apart in numbering would also slip through. To make sure that every model was trained and tested on exactly the same images, the split was saved once to a file that all models read, and our notebook generates the split again from its code and confirms that it matches this file.

Normalisation

All images are normalised with the ImageNet statistics, which EfficientNet-B0 requires and which we apply to the other models as well. Each image is converted to RGB, scaled to $[0, 1]$ and normalised per channel with the mean $(0.485, 0.456, 0.406)$ and the standard deviation $(0.229, 0.224, 0.225)$. Using the same values for the models trained from scratch means that the models differ only in the model itself, not in their input. Since these values are fixed constants, nothing is estimated from our own data, and no information from the test set can leak into training.

Augmentation (training set only)

Augmentation is applied only to the training set and is the same for all three models (Table 4). Because the photos are square, validation and test images only need to be resized, and no cropping is required at evaluation time.

Table 4: Training augmentation, applied anew every epoch.

Step	Setting
Random resized crop	80 to 100 % of the image area, output 224×224
Horizontal flip	probability 0.5
Rotation	up to $\pm 10^{\circ}$
Colour jitter	brightness, contrast and saturation 0.2; hue unchanged

2. Simple CNN (E1)

E1 is a small baseline CNN with three convolution blocks and 111,145 parameters. Each block consists of a 3×3 convolution, batch normalisation, ReLU and 2×2 max pooling, and the number of filters doubles from block to block (32, 64, 128) while the feature maps shrink from 224 to 28 pixels per side. Global average pooling and a small dense head then turn these maps into the nine class scores (Figure 3, Table 5).

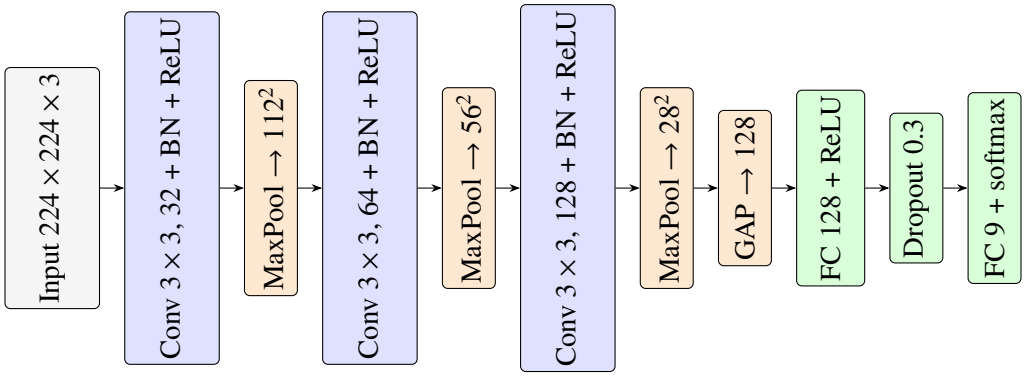


Figure 3: Architecture of E1 SimpleCNN. Blue: convolution, orange: pooling, green: fully connected part.

Table 5: E1 layers, output shapes and parameter counts.

Layer	Output shape	Parameters
Input	$224 \times 224 \times 3$	0
Conv 3×3 (32), BN, ReLU, MaxPool	$112 \times 112 \times 32$	928
Conv 3×3 (64), BN, ReLU, MaxPool	$56 \times 56 \times 64$	18,560
Conv 3×3 (128), BN, ReLU, MaxPool	$28 \times 28 \times 128$	73,984
Global average pooling	128	0
Dense 128, ReLU, Dropout 0.3	128	16,512
Dense 9 (logits)	9	1,161
Total		111,145

The shallow design of E1 makes it fast to train but limits what it can see. Its convolutions need no bias term, because the batch normalisation directly after them already adds its own shift. More importantly, after three blocks each value of the last feature map depends on only a 22×22 pixel patch of the input. Such a patch is enough to recognise a texture but not the outline of a bottle or a box, so E1 is likely to have trouble with classes that are recognised by their shape.

3. Complex CNN: MultiScaleCNN (E2)

E2 is our own complex CNN, built from blocks that look at the same input through three kernel sizes at once. The idea comes from a question raised in the lecture on modern CNNs, namely which kernel size is best. Following GoogLeNet, the lecture answers that combining several sizes can work better than choosing one [2, 6]. This answer suits our data well. A crumpled plastic bag is recognised by fine wrinkles and reflections, which a small kernel picks up, whereas a cardboard box is recognised by large flat faces and straight edges, which need a wider view.

Each multi-scale block therefore runs a 1×1, a 3×3 and a 5×5 convolution in parallel on the same input and stacks their outputs (Figure 4). Half of the output channels go to the 3×3 branch and a quarter to each of the other two. Compared with the Inception block in the lecture, we left out the extra 1×1 reductions and the pooling branch to keep the model simple.

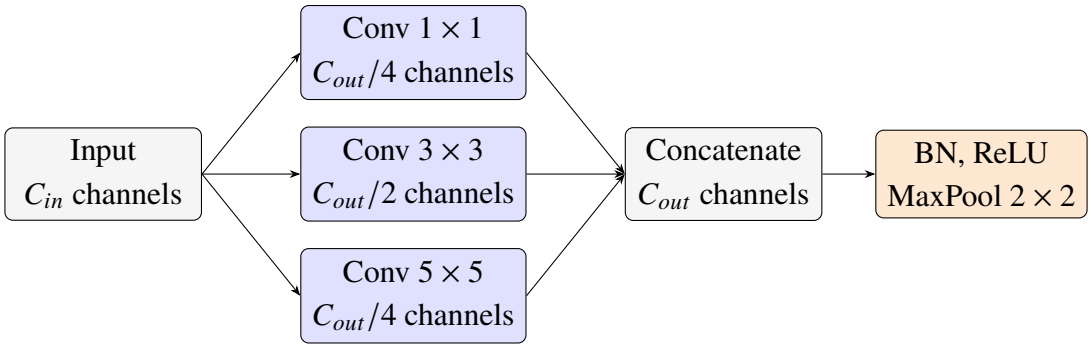


Figure 4: One multi-scale block. The three branches see the same input and are joined along the channel axis. Blocks 1 to 4 use C_{out} = 64, 128, 256 and 256.

The full network stacks four such blocks between a convolutional stem and the same kind of head as E1 (Figure 5). Every block is followed by max pooling, so the feature maps shrink from 112 to 7 pixels per side. With 1,230,377 parameters, E2 is about eleven times larger than E1 (Table 6).

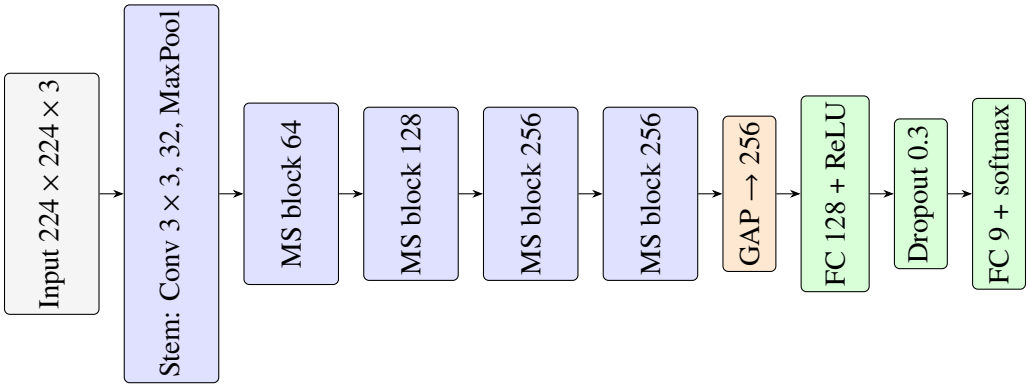


Figure 5: Architecture of E2 MultiScaleCNN. Every multi-scale (MS) block is followed by 2×2 max pooling, so the feature maps go from 112 to 7 pixels per side.

Table 6: E2 stages. Channel counts in brackets are the 1×1 / 3×3 / 5×5 branches.

Stage	Output shape	Parameters
Stem: Conv 3×3, 32, BN, ReLU, MaxPool	$112 \times 112 \times 32$	928
MS block 1 (16 / 32 / 16), MaxPool	$56 \times 56 \times 64$	22,656
MS block 2 (32 / 64 / 32), MaxPool	$28 \times 28 \times 128$	90,368
MS block 3 (64 / 128 / 64), MaxPool	$14 \times 14 \times 256$	360,960
MS block 4 (64 / 128 / 64), MaxPool	$7 \times 7 \times 256$	721,408
Global average pooling	256	0
Dense 128, ReLU, Dropout 0.3	128	32,896
Dense 9 (logits)	9	1,161
Total		1,230,377

The deeper design gives E2 a much wider view of each image than E1. Through its 5×5 branches, one value of the last block depends on a 154×154 pixel area of the input, compared with 22×22 in E1, so the network can make use of the shape of the whole item. At the same time, E2 differs from E1 in depth and width as well as in its multi-scale branches. If E2 performs better, the gain can therefore only be credited to the design as a whole, not to the multi-scale branches alone.

4. Transfer Learning and Fine-Tuning: EfficientNet-B0 (E3)

For transfer learning we chose EfficientNet-B0 [5], because it is accurate for its size and trains quickly on the free Colab GPU we had. With its original 1,000-class head the network has 5.3 million parameters and reaches 77.7 % top-1 accuracy on ImageNet-1K [11]. It is built from MBConv blocks, each of which widens the channels with a 1×1 convolution, applies a cheap depthwise 3×3 or 5×5 convolution with one filter per channel, reweights the channels with a squeeze-and-excitation step and narrows them again with another 1×1 convolution. We replaced the ImageNet head with Dropout(0.2) and a $1280 \rightarrow 9$ dense layer, which leaves 4,019,077 parameters (Figure 6).

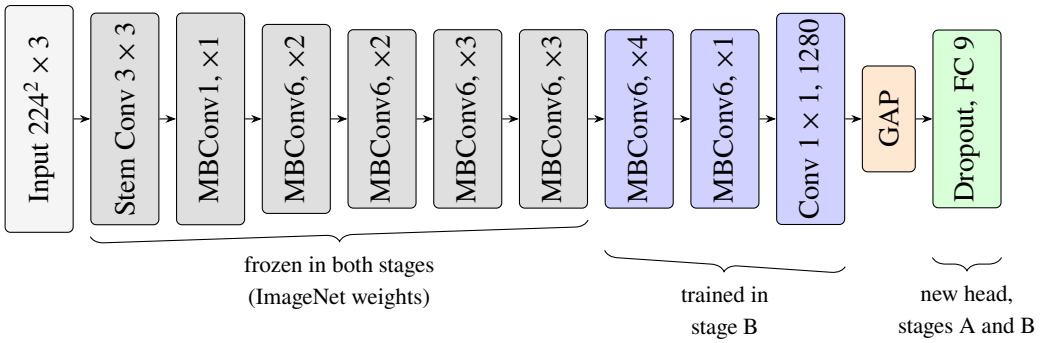


Figure 6: E3: EfficientNet-B0 with a new 9-class head. Grey blocks keep their ImageNet weights; the three blue blocks (`features[6:]`, all at 7×7 resolution) are fine-tuned in stage B.

E3 is trained in two stages that match the transfer and fine-tuning phases of the lecture [3] (Table 7). In stage A the whole backbone is frozen, including the running statistics of its batch normalisation layers, and only the new head learns. The backbone thus acts as a fixed feature extractor, and the head learns to read its 1,280 outputs. Stage B starts from the best stage-A checkpoint and additionally unfreezes the last three blocks, which hold the most task-specific features. The first six blocks stay frozen, both because they detect generic edges and textures that also suit waste photos and because this keeps each epoch short. Whichever stage reaches the higher validation macro-F1 becomes the final E3 model.

Table 7: The two training stages of E3.

	Stage A: feature extraction	Stage B: fine-tuning
Starts from	ImageNet weights, new random head	best stage-A checkpoint
Trainable part	head only	last 3 backbone blocks and head
Trainable parameters	11,529	3,167,269 (79 %)
Learning rate (AdamW)	10^{-3}	10^{-4}
Maximum epochs	10	15

Because E3 differs from E1 and E2 in its pretrained weights, its architecture and its training schedule, the comparison with them shows how transfer learning performs against training from scratch. It does not tell us which architecture is better.

5. Training Protocol

All three models were trained under one protocol whose settings were fixed before training began (Table 8). The two models trained from scratch share exactly the same values, and E3 differs only where its two stages require it.

Table 8: Training settings.

Setting	E1, E2	E3
Loss	cross-entropy, no class weights	same
Optimiser	AdamW, lr 10^{-3}, weight decay 10^{-4}	AdamW, lr 10^{-3} (stage A) and 10^{-4} (stage B), weight decay 10^{-4}
Learning-rate schedule	halved after 2 epochs without a better validation macro-F1	same, per stage
Batch size	32	32
Maximum epochs	40	10 + 15
Early stopping	5 epochs without a better validation macro-F1	same, per stage
Checkpoint kept	best validation macro-F1	best of both stages
Precision	mixed (16-bit) on GPU	same

Two settings were decided during the project, both on the basis of validation results only. E1 was first given a limit of 30 epochs, but its validation score was still rising at that point, so we raised the limit to 40 for both models trained from scratch and let the same run continue. We also considered weighting the loss by class frequency to help the small classes. After E2 had been trained, this no longer seemed necessary, because the deeper model recovered most of the small classes on its own: the validation recall of Textile Trash rose from 29 % with E1 to 51 % with E2, and that of Miscellaneous Trash from 30 % to 70 %.

Model selection relies on the validation macro-F1 rather than on accuracy. Both early stopping and the choice of the epoch to keep use this score, because with unequal classes it gives Textile Trash the same weight as Plastic.

We did not run a hyperparameter search, which is one of the limitations of this study.

A small search, with two learning rates and two values of one further setting per model, was planned but did not fit into the deadline. The values in Table 8 are therefore common defaults for AdamW and for fine-tuning, and they may not be the best choice for every model, E1 in particular.

Instead of a search, we repeated each training run to measure how much the results depend on chance. A single run is affected by the starting weights, the order of the batches, the augmentations that are drawn and the neurons that dropout switches off. Each model was therefore trained three times, with seeds 42, 43 and 44, and we report the mean and standard deviation (SD) of these runs. Since the split stays the same in all runs, the SD reflects training randomness only, not the choice of test images.

All training ran on Google Colab with an NVIDIA T4 GPU, where one epoch took about 40 seconds for E1 and E3 and about 45 seconds for E2. In practice, the slowest step was reading the images from Google Drive. The first epoch of each session took around 15 minutes for this reason, until we started copying the dataset to the Colab disk at the beginning of every session.

IV. Experimental Results

1. Metrics

We compare the models by accuracy and by the macro averages of precision, recall and F1, with macro-F1 as the main score. For a class k, precision P_k is the share of images predicted as k that really belong to k, and recall R_k is the share of the real k images that the model found. F1 combines the two, and the macro average gives every class the same weight:

$$F1_k = \frac{2\,P_k R_k}{P_k + R_k}, \qquad \text{Macro-F1} = \frac{1}{K}\sum_{k=1}^{K} F1_k. \tag{4}$$

Macro precision and macro recall are averaged in the same way. Accuracy, the share of all images classified correctly, is easier to read, but it can hide a weak small class; this is why macro-F1 decides the comparison.

2. Results on the Test Set

[TODO: fill Table 9 from the output of the submission notebook, then write the opening sentences of this subsection around the main result.]

Table 9: Test set (707 images), mean ± SD over seeds 42, 43 and 44. Precision, recall and F1 are macro averages.

Model	Accuracy	Precision	Recall	F1-score
E1 SimpleCNN	TBD	TBD	TBD	TBD
E2 MultiScaleCNN	TBD	TBD	TBD	TBD
E3 EfficientNet-B0 (fine-tuned)	TBD	TBD	TBD	TBD

[TODO: per-class test F1 from the submission notebook, with one paragraph on the hardest classes.]

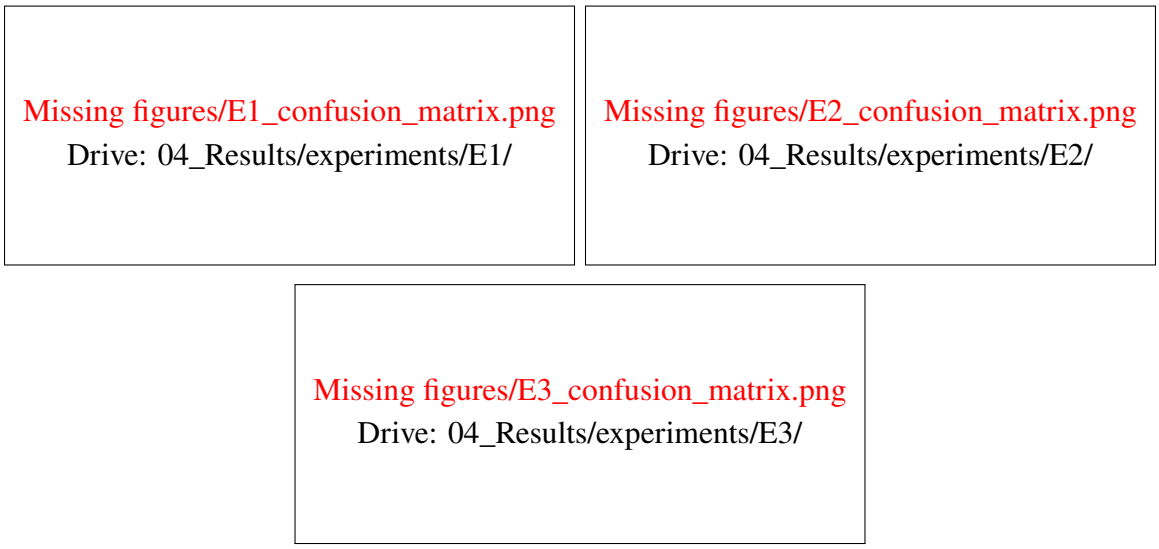


Figure 7: Test confusion matrices of the seed-42 runs (rows: true class, columns: predicted class).

3. Validation Results and Training Behaviour

On the validation set the three models rank clearly, with each step up adding roughly 0.1 macro-F1. Table 10 shows the scores of the seed-42 runs, which we used to select the checkpoints: E1 reached a macro-F1 of 0.650, E2 0.787 and E3 0.879.

Table 10: Validation set (719 images), seed-42 runs, selected checkpoints.

Model	Best epoch	Accuracy	Precision	Recall	F1-score
E1 SimpleCNN	33 of 38	0.656	0.706	0.636	0.650
E2 MultiScaleCNN	35 of 40	0.787	0.815	0.772	0.787
E3 EfficientNet-B0	25 of 25	0.880	0.894	0.871	0.879

[TODO: add the validation macro-F1 mean ± SD of the three seeds from each `selection.json`.]

The training curves in Figure 8 help to explain this ranking, because each model behaved differently during training. E1 kept improving slowly until about epoch 33, and its training and validation curves stayed close together throughout; it was not overfitting but simply lacked capacity. E2 showed the opposite pattern towards the end. After roughly epoch 30 its training loss kept falling, to 0.49 at the end, while the validation loss stayed around 0.64. This mild overfit did no harm, since we keep the epoch with the best validation score anyway.

E3 gave the most telling result. With only the new head trained, stage A already reached 0.810, which is more than our best model trained from scratch. Fine-tuning in stage B then raised the score to 0.879. By the end of stage B, however, the training accuracy was close to 100 % (training loss 0.02 against 0.42 on validation), so longer training would not have helped.

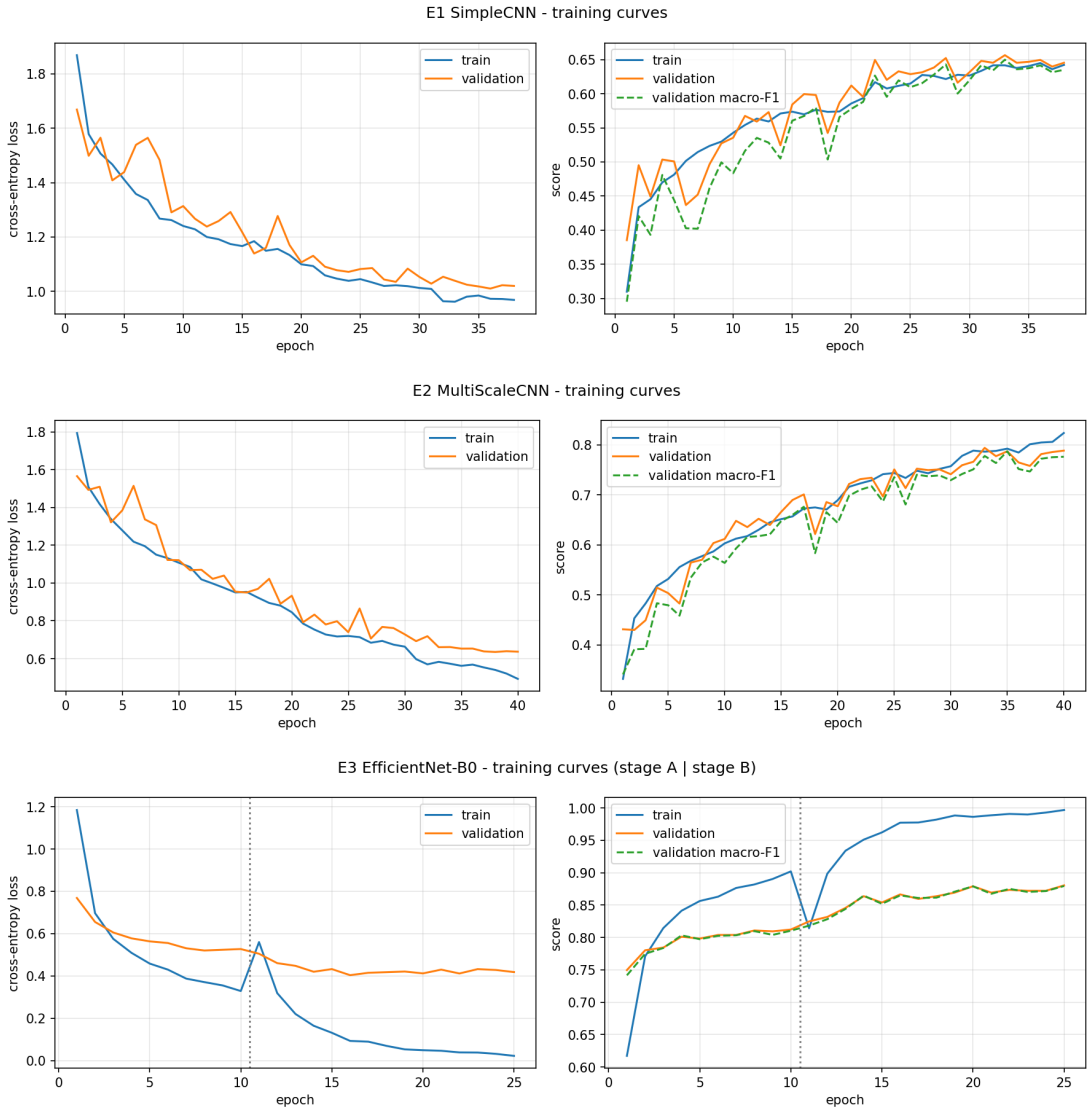


Figure 8: Training and validation loss, accuracy and macro-F1 per epoch for E1, E2 and E3 (seed 42).

4. Error Analysis

On the validation set, every model tended to fall back on Plastic when it was unsure, and the smaller the model, the stronger this habit. E1 predicted Plastic for 249 validation images, of which only 116 really were Plastic, so 54 % of all its errors were wrong Plastic predictions. This share dropped to 33 % for E2 and 31 % for E3. Plastic is the largest class and covers almost every shape, colour and degree of shine, which probably makes it the safest guess for a model that has not yet learned the other classes well. Table 11 lists the most frequent confusions of each model.

Table 11: Most frequent confusions on validation (true → predicted, number of images).

Model	Confusions
E1	Metal → Plastic 34, Miscellaneous → Plastic 31, Glass → Plastic 25, Cardboard → Plastic 18
E2	Plastic → Metal 23, Cardboard → Plastic 17, Glass → Plastic 14, Textile → Miscellaneous 13
E3	Miscellaneous → Plastic 9, Glass → Plastic 8, Metal → Plastic 6, Cardboard → Metal 5

The two classes that all models found hardest were Miscellaneous Trash and Textile Trash. E1 recognised only 30 % and 29 % of their images, E2 70 % and 51 %, and E3 71 % and 82 %. Both classes are collections of very different objects rather than one material, and Textile Trash is also the smallest class, so their low recall is not surprising.

[TODO: test error analysis: take the model with the best mean validation macro-F1, describe the failure modes of its misclassified test images grouped by confusion pair, and add a figure with example errors.]

V. Conclusion

[TODO: the findings below come from validation. Replace each number with the test mean ± SD and keep a finding only if the test results still support it.]

The fine-tuned EfficientNet-B0 (E3) was clearly the best of the three models, and the size of the gaps between them answers all three research questions. On the validation set E3 reached a macro-F1 of 0.879, compared with 0.787 for our multi-scale CNN (E2) and 0.650 for the simple CNN (E1).

RQ1. A simple CNN trained from scratch is too small for this task. It reaches a macro-F1 of 0.650 without overfitting, yet it misses most Textile Trash and Miscellaneous Trash items, which points to a lack of capacity rather than a lack of data.

RQ2. The multi-scale CNN is a clear improvement over the simple one. It raises macro-F1 by 0.137 and recovers much of the two hard classes. Since E2 is also deeper and wider than E1, however, we credit this gain to the design as a whole rather than to the multi-scale branches alone.

RQ3. ImageNet pretraining helps more than any architecture we designed ourselves. It adds about 0.09 macro-F1 on top of E2, and even the frozen pretrained features with only a new head (0.810) beat both models trained from scratch.

The errors that remain follow a common pattern across the models. They mostly confuse materials that look alike, such as glass, metal and plastic, or cardboard and plastic, they fall back on Plastic when unsure, and they struggle with the two mixed classes.

These results come with several limitations. First, photos of one object can still end up in different subsets when they straddle a block boundary or are far apart in numbering, so the test scores may be slightly optimistic. Second, we did not search for better hyperparameters, and E1 in particular might do better with other settings. Third, the standard deviation over three seeds covers training randomness only, since all runs share one split. Our comparisons also change more than one factor at a time: E2 differs from E1 in several ways, and E3 differs from both in pretraining and schedule. Finally, all images come from one setup with the same floor and camera angle, so accuracy on photos taken elsewhere, for example on a real sorting line, is likely to be lower.

For a practical sorting system with a small labelled dataset, our results favour starting from a pretrained network over designing a CNN from scratch. In our tests the pretrained model was the most accurate of the three and needed only 25 epochs to train. Before it could be used in the field, however, it would have to be retrained and tested on images from the place where it will run.

References

[1] N. T. K. Ngan, "Convolutional Neural Network (CNN)," lecture slides, Chapter 5.1, Deep Learning course, National Economics University, 2026.

[2] N. T. K. Ngan, "Modern Convolutional Neural Networks," lecture slides, Chapter 5.2, Deep Learning course, National Economics University, 2026.

[3] N. T. K. Ngan, "CNN Techniques," lecture slides, Chapter 5.3, Deep Learning course, National Economics University, 2026.

[4] S. Single, S. Iranmanesh, and R. Raad, "RealWaste: A novel real-life data set for landfill waste classification using deep learning," *Information*, vol. 14, no. 12, art. 633, 2023. `https://www.mdpi.com/2078-2489/14/12/633`. Dataset:

UCI Machine Learning Repository, https://archive.ics.uci.edu/dataset/908/realwaste.

[5] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. ICML*, 2019. arXiv:1905.11946.

[6] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," arXiv:1409.4842, 2014.

[7] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," arXiv:1502.03167, 2015.

[8] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[9] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019. arXiv:1711.05101.

[10] O. Russakovsky, J. Deng, H. Su, et al., "ImageNet large scale visual recognition challenge," arXiv:1409.0575, 2014.

[11] PyTorch, "torchvision `efficientnet_b0`, weights `IMAGENET1K_V1`," model documentation. https://docs.pytorch.org/vision/main/models/generated/torchvision.models.efficientnet_b0.html.